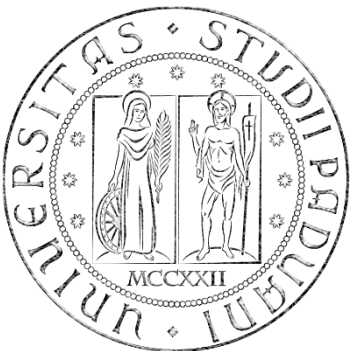


Biomedical Wearable Technologies for Healthcare and Wellbeing

Implementing UI

A.Y. 2024-2025

Giacomo Cappon

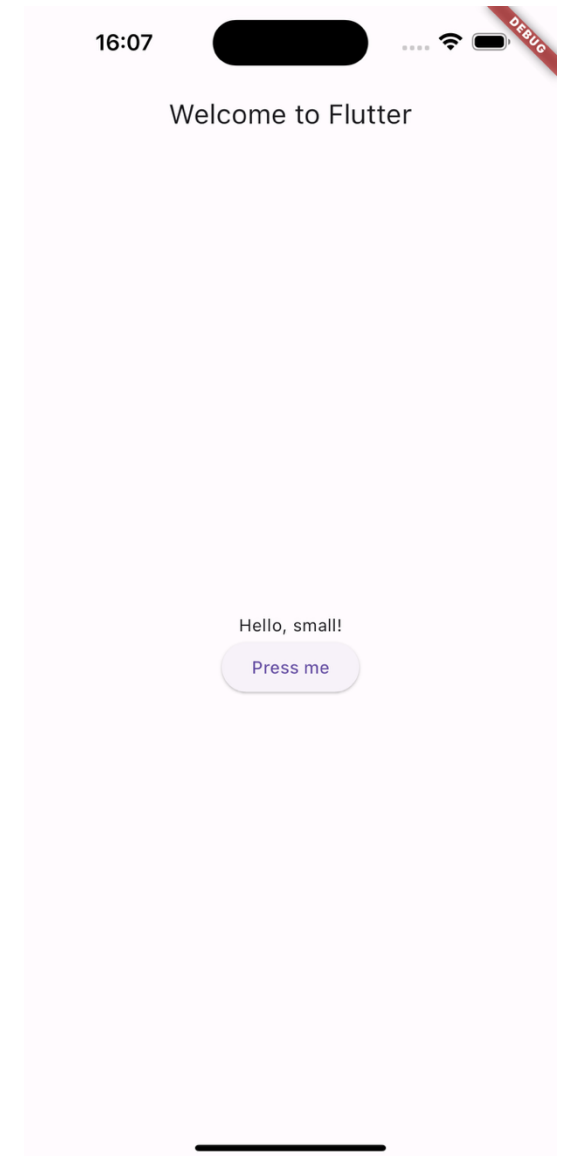
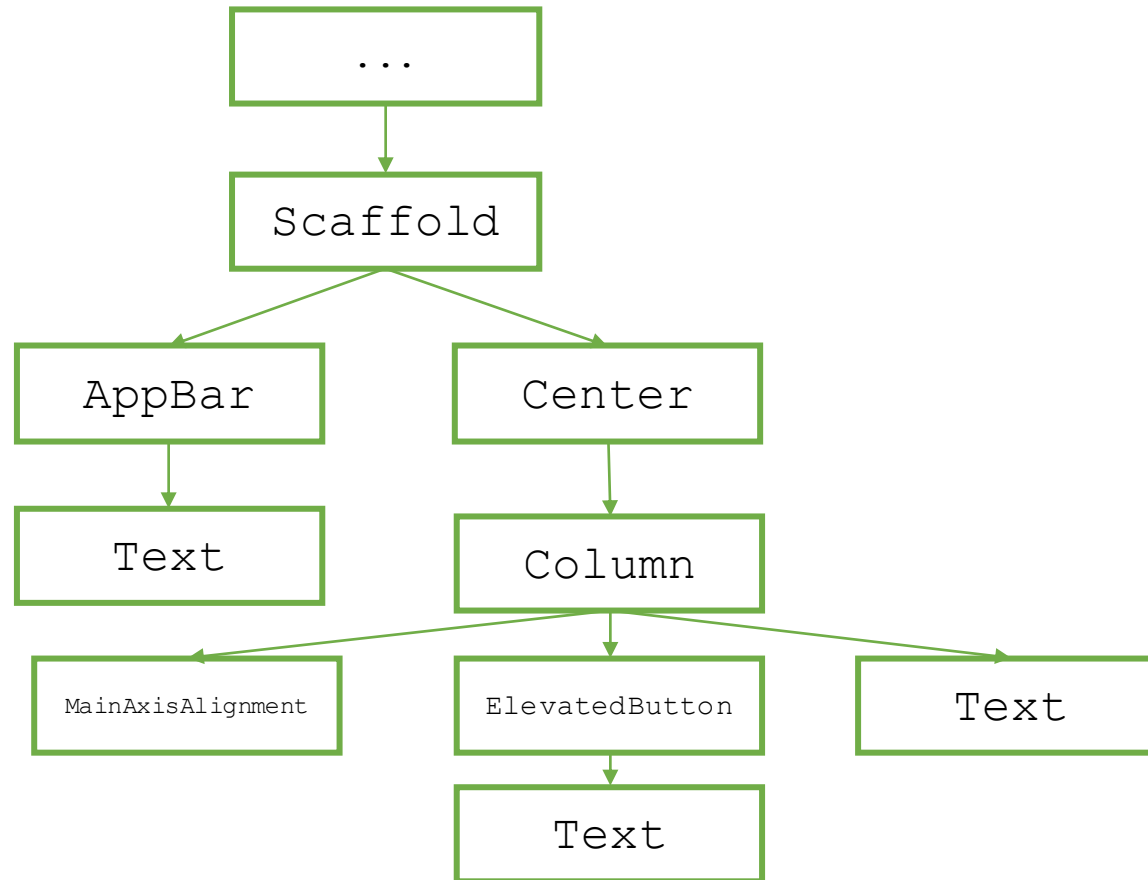


Outline

- **Recap**
- Building layouts in Flutter
- Basic examples
- Exercise
- Homework
- Resources

Recap: Widget tree

- UIs are represented in Flutter hierarchically through a Widget tree



Outline

- Recap
- **Building layouts in Flutter**
- Basic examples

- Exercise
- Homework
- Resources

Building layouts in Flutter

- Layouts are organized in a Widget tree
- But how a widget is sized and positioned somewhere?
- To answer this question, we need to fully understand this rule:

Constraints go down. Sizes go up. Parent sets position.



The set of 4 doubles:
minimum and maximum
height, minimum and
maximum width.



The set of 2
doubles: height
and width.

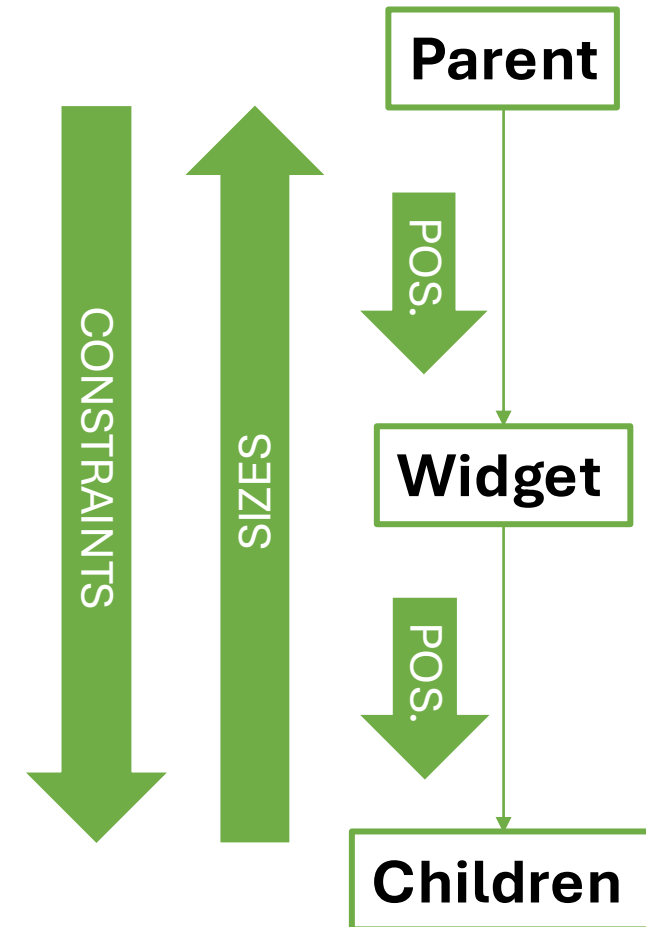


The set of 2
doubles: x and y.

Building layouts in Flutter

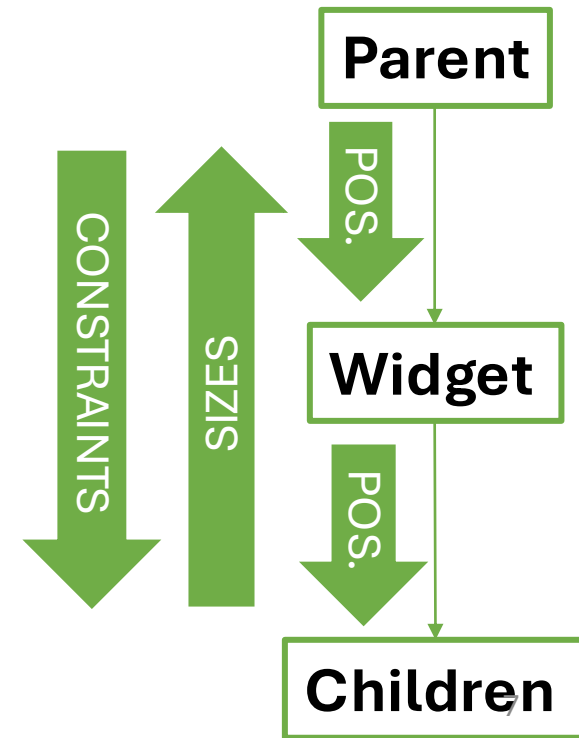
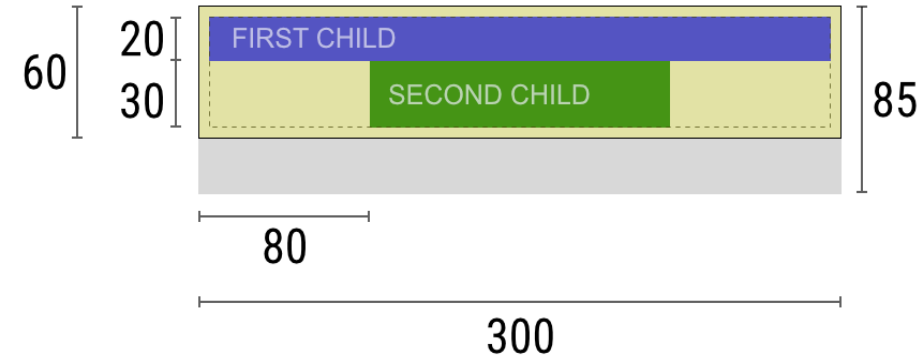
Constraints go down. Sizes go up. Parent sets position.

- A widget gets its constraints from its parent and tells its children what are the constraints (i.e., “**constraints go down**”).
- A widget then asks its children the sizes they want to be (i.e., “**sizes go up**”).
- A widget positions its children (i.e., “**parent sets position**”).



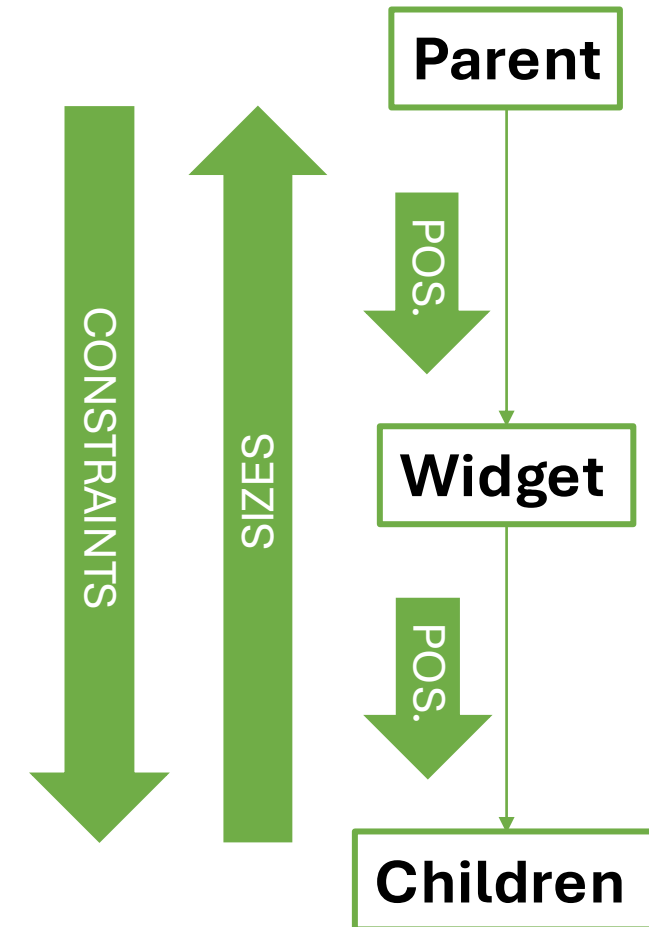
Negotiating constraints, sizes, and position

- **Widget:** “Hey parent, what are my constraints?”
- **Parent:** “You must be from 80 to 300 pixels wide, and 30 to 85 tall.”
- **Widget:** “Since I want to have 5 pixels of padding, then my children can have at most 290 pixels of width and 75 pixels of height.”
- **Widget:** “Hey first child, you must be from 0 to 290 pixels wide, and 0 to 75 tall.”
- **First child:** “OK, then I wish to be 290 pixels wide, and 20 pixels tall.”
- **Widget:** “Since I want to put my second child below the first one, this leaves only 55 pixels of height for my second child.”
- **Widget:** “Hey second child, You must be from 0 to 290 wide, and 0 to 55 tall.”
- **Second child:** “OK, I wish to be 140 pixels wide, and 30 pixels tall.”
- **Widget:** “Very well. My first child has position x: 5 and y: 5, and my second child has x: 80 and y: 25.”
- **Widget:** “Hey parent, I’ve decided that my size is going to be 300 pixels wide, and 60 pixels tall.”



Limitations

- A widget can decide its own size only within the constraints given to it by its parent: a widget **can't have any size it wants**.
- A widget **can't know and doesn't decide its own position in the screen**.
- It's impossible to precisely define the size and position of any widget without taking into consideration the tree as a whole.
- If a child wants a different size from its parent and the parent doesn't have enough information to align it, then **the child's size might be ignored**.



Outline

- Recap
- Building layouts in Flutter
- **Basic examples**
- Exercise
- Homework
- Resources

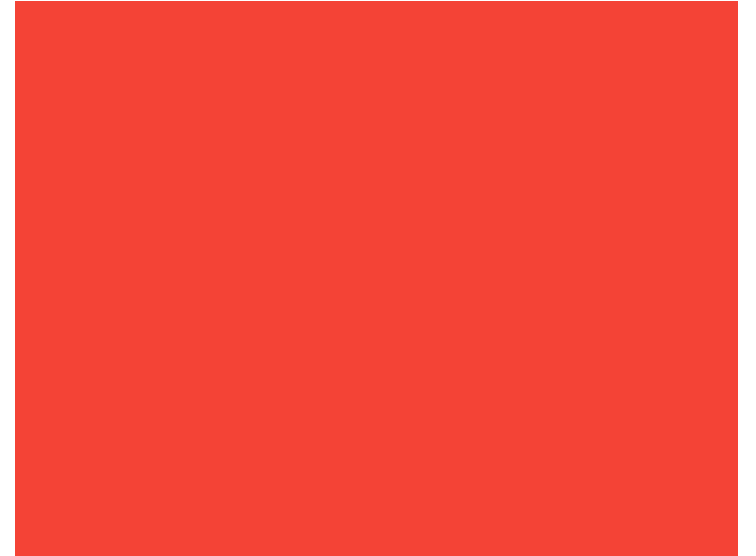
Dive into examples

- In the “Understanding constraints” article by Marcelo Glasberg <https://docs.flutter.dev/development/ui/layout/constraints> (you will also find the link in the Resources section of this presentation) there are a lot (29) of examples explaining how all of this works.
- Let's report the most interesting ones.

Example 1

```
Widget _example1() => Container(color:  
Colors.red);
```

- The screen is the parent of the Container, and it forces the Container to be exactly the same size as the screen.
- So the Container fills the screen and paints it red.



Example 2

```
Widget _example2() => Container(width:  
100, height: 100, color: red);
```

- The red `Container` wants to be 100×100 , but it can't, because the screen forces it to be exactly the same size as the screen.
- The `Container` fills the screen.



Example 3

```
Widget _example3() => Center(  
  child: Container(width: 100, height: 100,  
    color: red),  
);
```

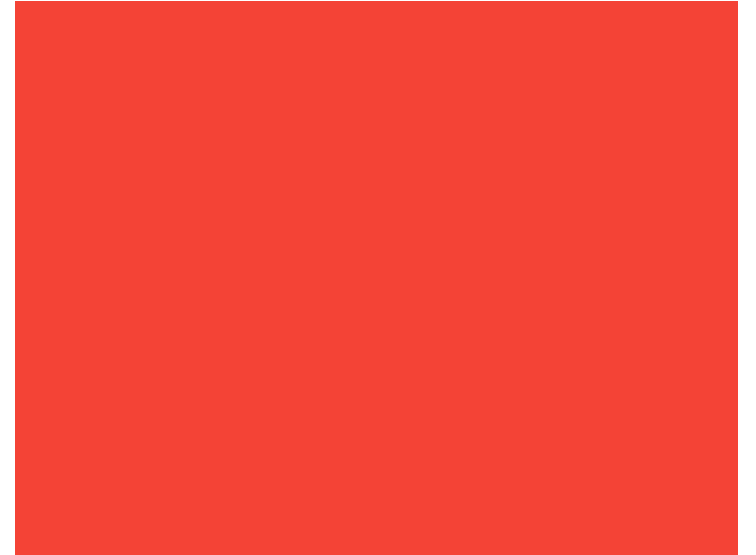
- The screen forces the `Center` to be exactly the same size as the screen, so the `Center` fills the screen.
- The `Center` tells the `Container` that it can be any size it wants, but not bigger than the screen. Now the `Container` can be 100×100 .



Example 6

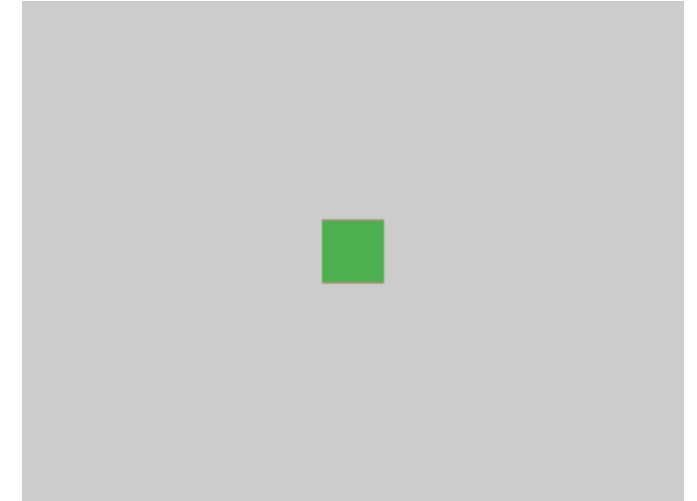
```
Widget _example6() => Center(  
  child: Container(color: red),  
)
```

- The screen forces the `Center` to be exactly the same size as the screen, so the `Center` fills the screen.
- The `Center` tells the `Container` that it can be any size it wants, but not bigger than the screen. Since the `Container` has no child and no fixed size, it decides it wants to be as big as possible, so it fills the whole screen.



Example 7

```
Widget _example7() => Center(  
  child: Container(  
    color: red,  
    child: Container(color: green, width: 30,  
height: 30),  
  ),  
);
```



- The screen forces the `Center` to be exactly the same size as the screen, so the `Center` fills the screen.
- The `Center` tells the red `Container` that it can be any size it wants, but not bigger than the screen. Since the red `Container` has no size but has a child, it decides it wants to be the same size as its child.
- The red `Container` tells its child that it can be any size it wants, but not bigger than the screen.
- The child is a green `Container` that wants to be 30×30 . Given that the red `Container` sizes itself to the size of its child, it is also 30×30 . The red color isn't visible because the green `Container` entirely covers the red `Container`.

Example 14

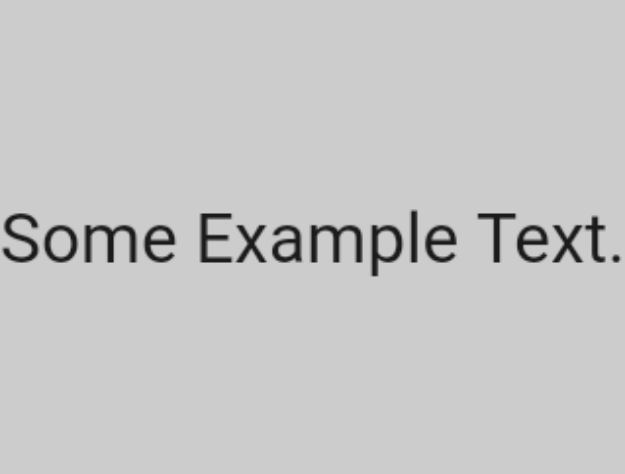
```
Widget _example14() => UnconstrainedBox(  
  child: Container(color: red, width: 4000,  
height: 50),  
);
```

- The screen forces the `UnconstrainedBox` to be exactly the same size as the screen, and `UnconstrainedBox` lets its child `Container` be any size it wants.
- Unfortunately, in this case the `Container` is 4000 pixels wide and is too big to fit in the `UnconstrainedBox`, so the `UnconstrainedBox` displays the much dreaded “overflow warning”.



Example 18

```
Widget _example18() => const FittedBox(  
  child: Text('Some Example Text.'),  
);
```

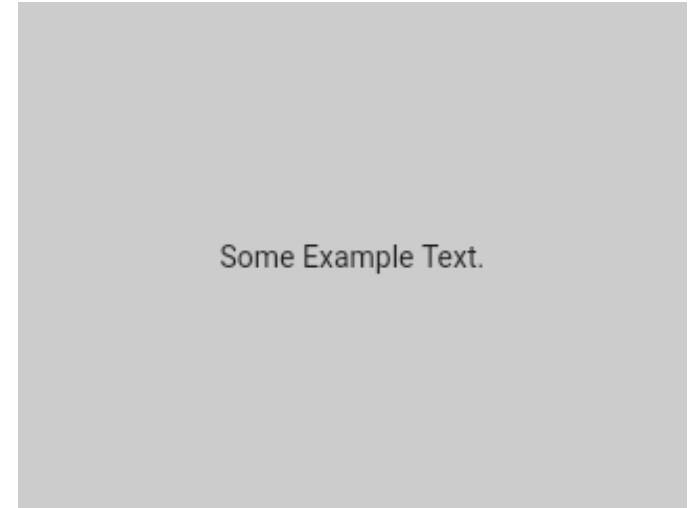


Some Example Text.

- The screen forces the `FittedBox` to be exactly the same size as the screen. The `Text` has some natural width (also called its intrinsic width) that depends on the amount of text, its font size, and so on.
- The `FittedBox` lets the `Text` be any size it wants, but after the `Text` tells its size to the `FittedBox`, the `FittedBox` scales the `Text` until it fills all of the available width.

Example 19

```
Widget _example19() => const Center(  
  child: FittedBox(  
    child: Text('Some Example Text.'),  
  ),  
);
```



- But what happens if you put the `FittedBox` inside of a `Center` widget? The `Center` lets the `FittedBox` be any size it wants, up to the screen size.
- The `FittedBox` then sizes itself to the `Text`, and lets the `Text` be any size it wants. Since both `FittedBox` and the `Text` have the same size, no scaling happens.

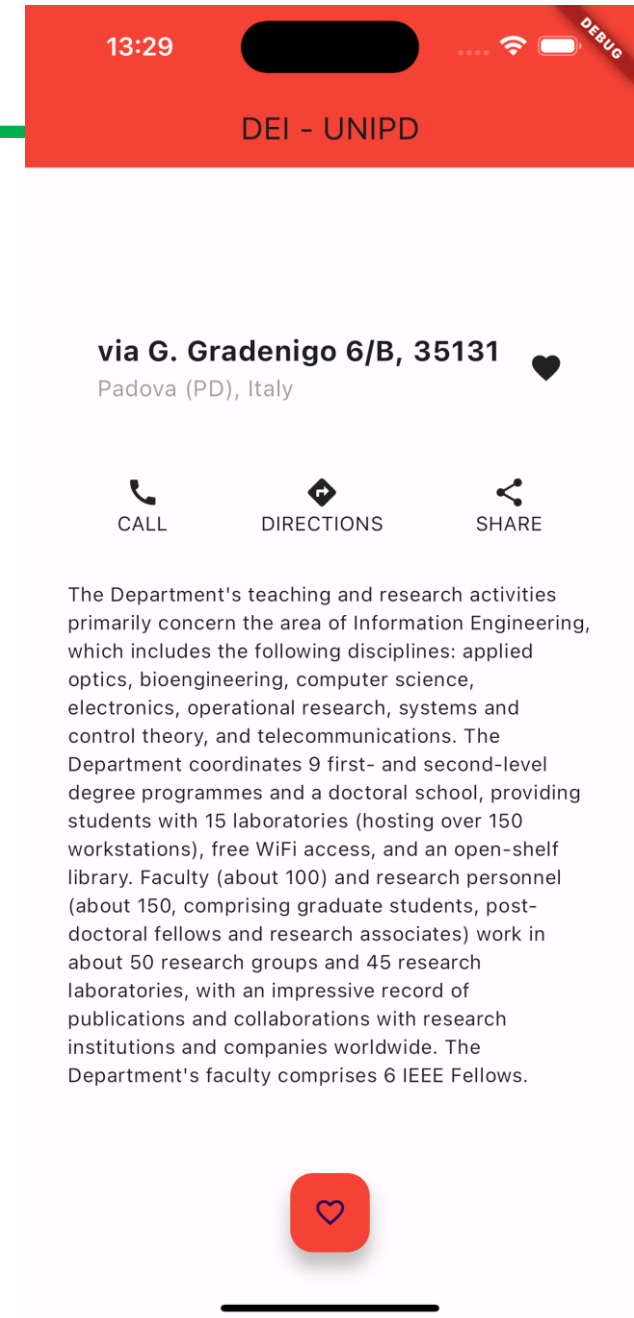
Outline

- Recap
- Building layouts in Flutter
- Basic examples
- **Exercise**
- Homework
- Resources

Exercise

➤ Exercise 05.01

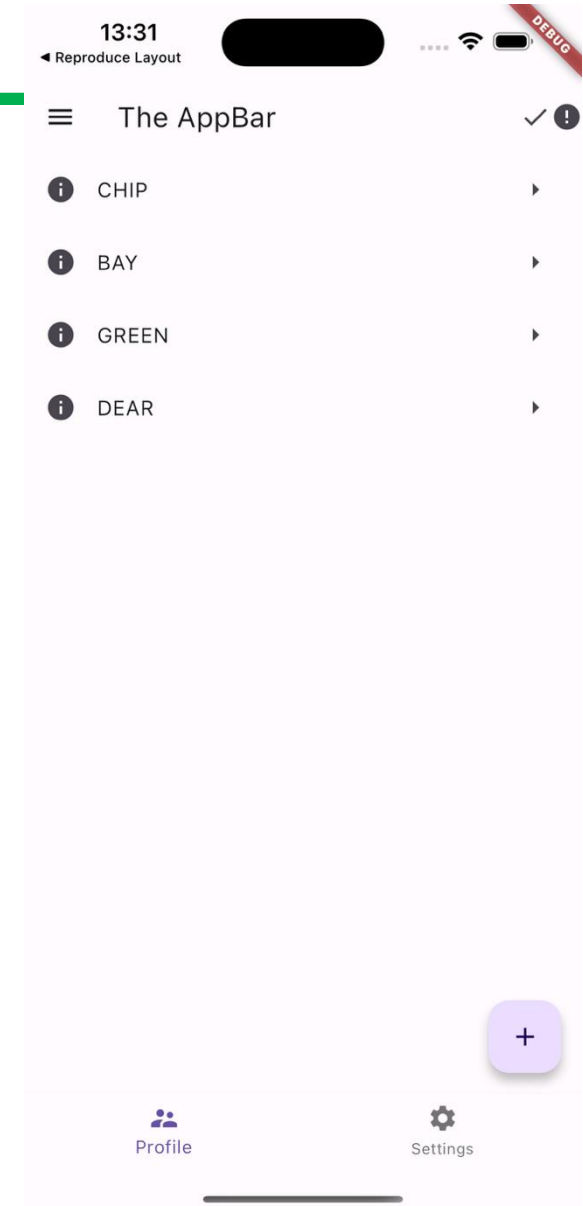
- Create a new project 'reproduce_layout'
- Reproduce, as close as possible, the layout on the right.
- Hint: I used the following widgets: AppBar, Text, Container, Icon, SizedBox, Column, Row, Colors, and some others...



Exercise

➤ Exercise 05.02

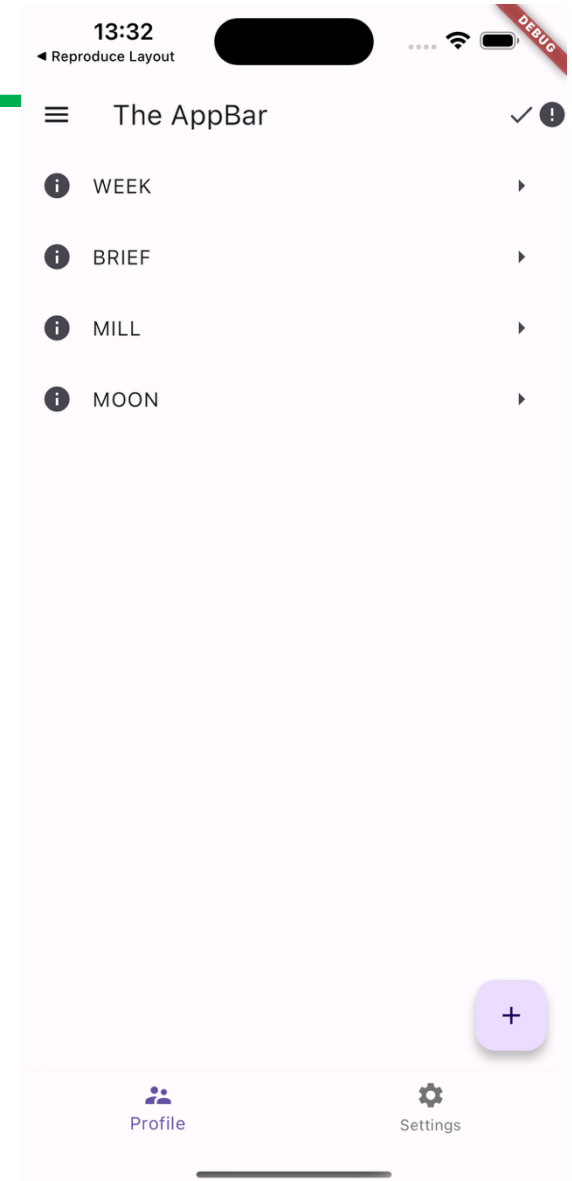
- Create a new project 'exercise_listview'
- Copy the code of main.dart of 'scaffolding' into the main.dart of 'exercise_view'
- Modify the ListView so that the app looks like the app on the right (hint: I used Icons.info)



Exercise

➤ Exercise 05.03

- Install the english_words package into the 'exercise_listview' app
- Starting from the code of exercise 05.02, modify each ListTile in order to have as a title a random word in uppercase generated using the english_words package just installed



Outline

- Recap
- Building layouts in Flutter
- Basic examples
- Exercise
- **Homework**
- Resources

Homework

- Get familiar with UI constraints
- Get fluent in writing simple UIs

Outline

- Recap
- Building layouts in Flutter
- Basic examples
- Exercise
- Homework
- **Resources**

Resources

- Code repository of today's lesson and exercises solution
 - https://github.com/gcappon/bwthw/tree/master/lesson_08-implementing_ui
- Material Widgets for UI
 - <https://docs.flutter.dev/development/ui/widgets/material>
- (Some) Widget for UI Catalog
 - <https://docs.flutter.dev/development/ui/widgets>
- Flutter Widget list
 - <https://docs.flutter.dev/reference/widgets>
- Understanding constraints
 - <https://docs.flutter.dev/development/ui/layout/constraints>
- Layouts in Flutter
 - <https://docs.flutter.dev/development/ui/layout>